ScoreProcessor File Handling Utility (Python + Pytest)

This solution demonstrates robust file handling using try-except-else-finally, custom error handling for missing files and invalid data formats, and automated testing with pytest.

score_processor.py

```
class ScoreProcessor:

    def process_score_file(self, file_path: str) -> int:
        try:
            with open(file_path, "r") as file:
                content = file.read().strip()
                score = int(content)

        except FileNotFoundError:
            print(f"Error: File '{file_path}' was not found.")
            raise

        except ValueError:
            print("Error: File contains invalid data. "
                  "Expected an integer value.")
            raise

        else:
            result = score * 10
            print("Data processed successfully")
            return result

        finally:
            print("File cleanup completed")
```

test_score_processor.py

```
import pytest
from score_processor import ScoreProcessor


def test_successful_calculation(tmp_path):
    file_path = tmp_path / "score.txt"
    file_path.write_text("8")

    processor = ScoreProcessor()

    result = processor.process_score_file(str(file_path))

    assert result == 80


def test_missing_file():
    processor = ScoreProcessor()

    with pytest.raises(FileNotFoundError):
        processor.process_score_file("missing_file.txt")


def test_invalid_data_format(tmp_path):
    file_path = tmp_path / "invalid.txt"
    file_path.write_text("ABC")

    processor = ScoreProcessor()

    with pytest.raises(ValueError):
        processor.process_score_file(str(file_path))
```